Arts et Métiers Sciences et Technologies

PJE : Optimisation de la stratégie de stockage d’énergie électrique par CAES dans le contexte du marché européen de l’électricité

-

Mehdi CHTOUROU

SOMMAIRE

I. Introduction générale

II. Présentation générale du marché européen de l'électricité

III. Modélisation du système CAES

IV. Stratégie de stockage dans le contexte du marché de l'électricité

V. Modélisation des prix de l'électricité

VI. Mise en lien des prix et de la stratégie : algorithmes de pilotage

VII. Observations et analyse des résultats

VIII. Conclusion et perspectives

IX. Annexes

X. Bibliographie

I. Introduction générale

La transition énergétique européenne s'accompagne d'une transformation profonde du système électrique. L'augmentation rapide de la part des énergies renouvelables variables, telles que l'éolien et le solaire, modifie les équilibres traditionnels entre production et consommation. Cette évolution se traduit par une volatilité accrue des prix de l'électricité sur les marchés de gros, avec des épisodes de prix très bas, voire négatifs, alternant avec des périodes de tension marquées.

Dans ce contexte, le stockage d'énergie apparaît comme un outil central pour apporter de la flexibilité au système électrique. Parmi les technologies disponibles, le stockage par air comprimé (Compressed Air Energy Storage – CAES) constitue une solution de stockage massif capable de fournir des puissances élevées sur des durées longues. Toutefois, la viabilité économique d'un système CAES ne dépend pas uniquement de ses performances techniques, mais aussi, et surtout, de la stratégie de pilotage adoptée vis-à-vis des marchés de l'électricité.

L'objectif de ce travail est d'étudier comment une stratégie de stockage et de déstockage peut être optimisée pour un système CAES, en s'appuyant sur des modèles de prévision des prix de l'électricité et sur des règles de décision adaptées. La problématique centrale est la suivante :

Comment relier de manière cohérente le fonctionnement physique d'un système CAES, la dynamique des prix de l'électricité sur le marché européen et une stratégie algorithmique de pilotage permettant d'optimiser l'arbitrage économique ?

II. Présentation générale du marché européen de l'électricité

Le marché européen de l'électricité occupe une place clé dans la vie et l'économie du continent. En effet, il représentait environ 285Mds€ en 2024 et devrait croître jusqu'à atteindre 395Mds€ en 2033. Le volume d'électricité généré en 2023 était de 2572 TWh. La forte pénétration des énergies renouvelables dans ce marché est aussi à noter, étant responsables de 43,2% de la production en 2024.

Le marché européen de l'électricité est organisé autour de plusieurs horizons temporels, allant des marchés à terme (notamment pour les solutions de stockage très importantes telles que les barrages, etc.) jusqu'aux mécanismes d'équilibrage en temps réel. Le marché day-ahead joue un rôle central dans la formation du prix de référence, tandis que les marchés intraday permettent d'ajuster les positions en fonction des écarts entre prévisions et réalisations effectives.

Les prix observés sur ces marchés sont le résultat d'un mécanisme de coût marginal, fortement influencé par : la structure du parc de production, la disponibilité des moyens pilotables, la demande, ainsi que les conditions météorologiques. L'essor des énergies renouvelables, caractérisées par des coûts marginaux très faibles mais une production intermittente, accentue la variabilité des prix et crée des opportunités d'arbitrage temporel.

Du point de vue du stockage d'énergie, ce contexte présente à la fois des opportunités et des contraintes. Les écarts de prix entre périodes creuses et périodes de pointe constituent un levier économique potentiel, mais l'incertitude associée aux prix futurs rend la prise de décision délicate. Une stratégie de stockage efficace doit donc être capable de s'adapter à un environnement de marché volatil et imparfaitement prévisible.

III. Modélisation du système CAES

a. Modèle Adiabatique

Le modèle physique du système CAES adiabatique constitue le socle de la chaîne de simulation et d'optimisation développée dans ce travail. Il permet de représenter de manière cohérente la dynamique de stockage d'énergie sous forme d'air comprimé, en traduisant une consigne de puissance électrique — positive en phase de charge, négative en phase de décharge — en une évolution réaliste de l'état interne du stockage, tout en respectant les contraintes techniques et énergétiques du système.

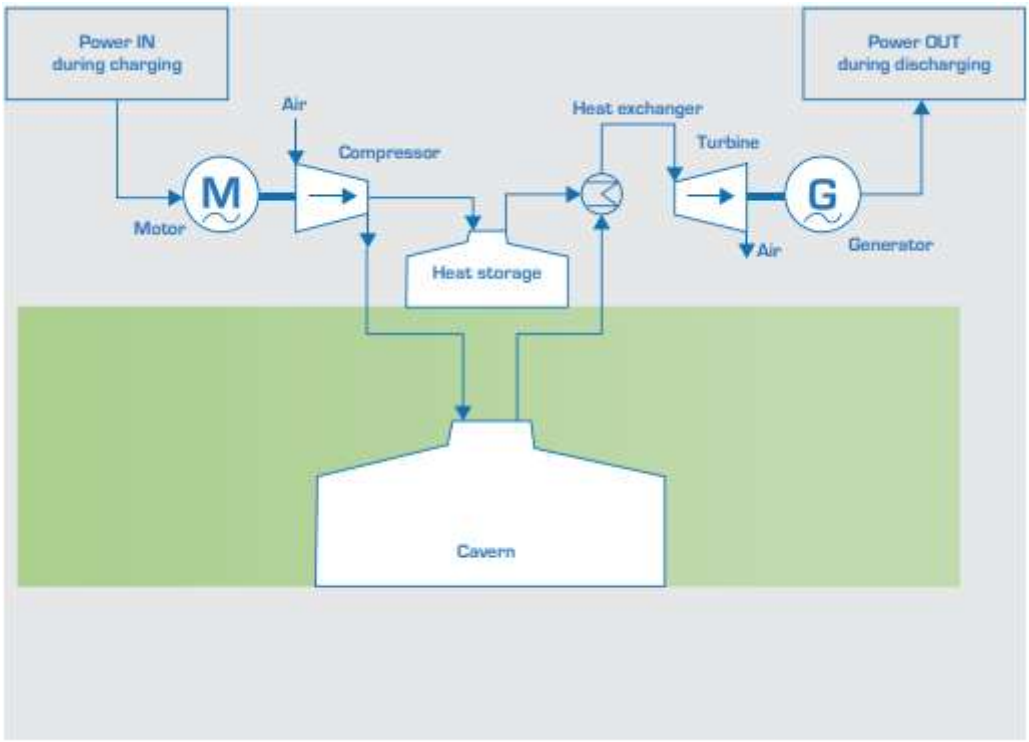


Figure 1a : Modèle CAES adiabatique

Le code simuler_caes_adiabatique implémente un modèle légèrement simplifié (rendements constants, pertes moyennées, et idem pour le cas isotherme), mais physiquement cohérent, du fonctionnement d'un stockage CAES en régime adiabatique. Les paramètres du système sont regroupés dans une structure dédiée (ParametresCAESAdiabatique), ce qui permet de centraliser l'ensemble des hypothèses physiques et techniques. Cette

structure inclut notamment le volume de la caverne, les pressions minimale et maximale admissibles, la température ambiante de référence, le coefficient adiabatique de l'air, ainsi que les caractéristiques énergétiques du système : capacité maximale de stockage (200 MWh), puissances maximales du compresseur (70 MW) et de la turbine (120 MW), rendements de charge et de décharge, et pertes horaires représentant les phénomènes de dissipation et d'auto-décharge du stockage.

La fonction de simulation prend en entrée un index temporel discret, une série temporelle de consignes de puissance, et les paramètres du système. Le pas de temps est calculé automatiquement à partir de l'index, ce qui rend le modèle adaptable à différentes résolutions temporelles. À chaque pas de temps, l'état du stockage est mis à jour de manière récursive à partir de l'état précédent. Les variables suivies incluent la pression, la température, la masse d'air contenue dans la caverne, ainsi que l'énergie stockée exprimée en MWh.

Les pertes énergétiques sont modélisées sous la forme d'un taux horaire appliqué à l'énergie stockée, traduisant les effets combinés des fuites, des échanges thermiques non récupérés et des pertes mécaniques : $E_{k+1} = E_k * (1 - \lambda \Delta t)$ avec $\lambda = 5 * 10^{-5} h^{-1}$. Dans le cas adiabatique, le coefficient de pertes est plus faible car le système implique une meilleure gestion thermique et une récupération de chaleur idéale. La consigne de puissance est ensuite interprétée : une consigne positive correspond à une phase de compression, durant laquelle la puissance électrique appelée est limitée par la puissance maximale du compresseur et pondérée par le rendement de charge : $E_{k+1} = E_k + \eta_{charge} * P_{comp} * \Delta t \leq E_{max}\ avec\ \eta_{charge} = 0{,}85$. L'énergie effectivement stockée est alors incrémentée, dans la limite de la capacité maximale du système. À l'inverse, une consigne négative correspond à une phase de détente, limitée par la puissance maximale de la turbine et par l'énergie disponible dans le stockage, avec une prise en compte explicite du rendement de décharge : $E_{k+1} = E_k - \eta_{décharge} * P_{turb} * \Delta t \geq 0\ avec\ \eta_{décharge} = 0{,}88$.

Dans ce modèle adiabatique simplifié, la pression dans la caverne n'est pas calculée à partir des équations complètes de la thermodynamique adiabatique, mais est reconstruite a posteriori à partir du niveau d'énergie stockée $P_k = P_{min} + \frac{E_k}{E_{max}} * (P_{max} - P_{min})\ avec\ P_{min} = 1\ bar\ et\ P_{max} = 70\ bars$. Un taux de remplissage énergétique est défini comme le rapport entre l'énergie courante et la capacité maximale, et la pression est interpolée linéairement entre les pressions minimale et maximale admissibles. Cette approche permet de conserver une cohérence physique globale tout en limitant la complexité numérique du modèle.

Ce modèle joue un rôle central dans l'étude : il garantit que les stratégies de pilotage testées restent compatibles avec les limites physiques du système et permet d'évaluer l'impact réel des décisions de charge et de décharge sur l'état du stockage, la pression interne et les flux de puissance échangés avec le réseau électrique.

b. Modèle Isotherme

En complément du modèle adiabatique, un second modèle de stockage CAES en régime isotherme a été développé afin de comparer différentes hypothèses thermodynamiques et d'évaluer leur influence sur la dynamique énergétique du système. Le modèle isotherme repose sur l'hypothèse d'une température constante dans la caverne, correspondant à un échange thermique parfait avec l'environnement.

Le code simuler_caes_isotherme implémente ce modèle à partir d'une structure de paramètres dédiée (ParametresCAESIsotherme), qui regroupe les caractéristiques géométriques et énergétiques du système : volume de la caverne, pressions minimale et maximale, température constante, constante des gaz parfaits, puissances maximales du compresseur et de la turbine, rendements de charge et de décharge, capacité maximale de stockage, énergie initiale et pertes horaires.

La simulation repose sur un suivi explicite de l'énergie stockée, de la pression et de la masse d'air contenue dans la caverne. À l'initialisation, la pression est fixée à la valeur minimale admissible, la masse d'air est calculée à partir de l'équation des gaz parfaits en régime isotherme, et l'énergie stockée est initialisée à une valeur donnée. Comme pour le modèle adiabatique, le pas de temps est déduit automatiquement de l'index temporel.

À chaque pas de temps, l'énergie stockée est d'abord corrigée des pertes horaires, traduisant les phénomènes de dissipation du système : $E_{k+1} = E_k * (1 - \lambda \Delta t)$ avec $\lambda = 10^{-4} h^{-1}$. La consigne de puissance est ensuite appliquée. En phase de charge, la puissance électrique est limitée par la puissance maximale du compresseur et affectée du rendement de charge, ce qui permet de déterminer l'énergie effectivement ajoutée au stockage : $E_{k+1} = E_k + \eta_{charge} * P_{comp} * \Delta t \leq E_{max}\ avec\ \eta_{charge} = 0{,}85$. En phase de décharge, la puissance fournie au réseau est limitée par la puissance maximale de la turbine et par l'énergie disponible, avec une correction par le rendement de décharge : $E_{k+1} = E_k - \eta_{décharge} * P_{turb} * \Delta t \geq 0\ avec\ \eta_{décharge} = 0{,}88$.

Contrairement à un modèle thermodynamique isotherme complet basé sur les intégrales logarithmiques du travail de compression et de détente, le lien entre énergie stockée et pression est ici simplifié. La pression dans la caverne est reconstruite à partir du niveau d'énergie stockée via un taux de remplissage énergétique, interpolé linéairement entre les pressions minimale et maximale : $P_k = P_{min} + \frac{E_k}{E_{max}} * (P_{max} - P_{min})\ avec\ P_{min} = 1\ bar\ et\ P_{max} = 70\ bars$. Cette approche permet de conserver une cohérence globale entre énergie, pression et masse d'air, tout en assurant une grande robustesse numérique et une facilité d'intégration dans une chaîne d'optimisation.

La masse d'air contenue dans la caverne est ensuite recalculée à chaque pas de temps à partir de la pression obtenue, du volume de la caverne et

de la température constante, conformément à l'équation des gaz parfaits : $m_k = \frac{P_k V}{rT}$. Le modèle fournit ainsi une description complète de l'état du stockage à chaque instant : énergie stockée, pression interne, masse d'air, ainsi que les flux de puissance associés à la compression, à la détente et aux échanges avec le réseau.

Ce modèle isotherme, bien que simplifié, constitue un outil pertinent pour comparer les performances énergétiques et opérationnelles du stockage CAES sous différentes hypothèses thermiques. Il permet notamment d'analyser l'impact des rendements, des pertes et des stratégies de pilotage sur l'évolution du stockage, tout en garantissant la faisabilité physique des scénarios simulés. Le modèle isotherme représente un cas idéal permettant d'évaluer le potentiel maximal du CAES, tandis que le modèle adiabatique est plus représentatif des systèmes industriels actuels.

IV. Stratégie de stockage dans le contexte du marché de l'électricité

La stratégie de stockage vise à exploiter les fluctuations des prix de l'électricité afin de maximiser un gain économique. Dans sa forme la plus simple, cette stratégie repose sur un principe d'arbitrage : stocker lorsque les prix sont bas et restituer l'énergie lorsque les prix sont élevés.

Toutefois, cette logique intuitive se heurte à plusieurs difficultés. Les prix futurs ne sont pas connus avec certitude, et les décisions doivent être prises sur la base de prévisions imparfaites. Un écart, même infinitésimal, peut alors engendrer une prise de décision contraire aux conditions présentes. De plus, le stockage est soumis à des contraintes techniques (capacités, rendements, pertes) qui limitent la flexibilité réelle du système.

Dans ce projet, deux mécanismes de prise de décision ont été explorés. Le premier est le modèle de prise de décision par seuil fixe, et le deuxième est

le modèle de décision par seuil glissant, adoptant la méthode de la moyenne mobile.

V. Modélisation des prix de l'électricité

La modélisation des prix de l'électricité constitue un élément clé du dispositif. Après être passé par différentes étapes de conceptualisation de la modélisation, en ayant d'abord commencé par le développement d'une solution reposant sur le machine learning avec le modèle Gradient Boosting Regressor, j'ai ensuite travaillé sur le modèle de volatilité stochastique. Deux approches complémentaires ont été développées, puis réunies dans un même algorithme afin de comparer l'impact de l'incertitude et de la volatilité sur la stratégie de stockage.

Le code Pricing_Elec_v4 met en œuvre une approche hybride combinant apprentissage automatique et modélisation stochastique de la volatilité. Les prix historiques sont d'abord utilisés pour entraîner un modèle de type Random Forest Regressor, basé sur des variables retardées (lags). Le choix de passer du modèle Gradient Boosting Regressor au modèle Random Forest Regressor est dû au fait que ce dernier est plus adapté aux données utilisées (variations intraday, saisonnalité, tendance, etc), plus robuste au bruit que nous traitons ensuite, moins sensible à l'overfitting, etc. Le bruit du modèle est ensuite analysé à l'aide d'un modèle GARCH (via le module arch), permettant de simuler une volatilité conditionnelle réaliste. Les prix futurs sont ainsi générés comme la somme d'une tendance prédite par le modèle de machine learning et d'un bruit stochastique dont l'amplitude dépend de la volatilité estimée. Il est important de noter que la modélisation est faite de cette manière car nous manquons de certaines données à intégrer en temps réel, comme le ferait un desk de trading, pour pouvoir utiliser un modèle 100% ML et ne pas avoir à avoir recours à une volatilité artificielle pouvant fausser certaines tendances sur le long terme.

Cette approche présente l'avantage d'introduire une part d'aléa cohérente avec le comportement réel des marchés de l'électricité, rendant les trajectoires de prix plus réalistes du point de vue économique.

L'utilisation conjointe de ces deux modèles permet de comparer des stratégies optimales dans des contextes de prévision plus ou moins incertains.

VI. Mise en lien des prix et de la stratégie : algorithmes de pilotage

Le lien entre les prix de marché et le fonctionnement du CAES est assuré par les algorithmes de décision et de pilotage. Ces algorithmes traduisent une information économique (le prix de l'électricité) en une consigne de puissance compatible avec le modèle physique du stockage.

J'ai d'abord développé une première méthode, dans le code Modele_Decision_Seuil, qui s'est avérée être plus utile comme test que comme modèle de décision : fixer un seuil de prix, autour duquel la décision était prise de stocker l'énergie produite et acheter de l'énergie électrique sur le marché, ou bien de consommer l'énergie produite. Cependant, cette méthode s'est avérée être très limitée car elle ne prenait pas du tout en compte les saisonnalités et tendances. Par exemple : l'hiver et l'été les prix augmentent avec la demande, le modèle par seuil fixe n'aurait alors fait qu'utiliser l'énergie stockée, dépassant à un moment les limites physiques du système.

Le code Modele_Decision_Seuil_Glissant implémente une stratégie basée sur un seuil adaptatif, défini comme la moyenne mobile des prix sur une fenêtre donnée. À chaque instant, si le prix est inférieur à ce seuil, le système entre en phase de charge ; s'il est supérieur, il passe en phase de décharge. Cette approche est simple et intuitive, mais elle conduit à un grand nombre de changements de stratégie, reflétant une forte sensibilité aux fluctuations de court terme.

Le pilotage implémenté dans Principal_Seuil_Glissant_Adiabatique/Isotherme repose sur les prix prévus par l'algorithme Pricing_Elec_v4, qui sont utilisés pour calculer une trajectoire optimale de charge et de décharge, en tenant compte des contraintes du système CAES. Les consignes obtenues sont ensuite injectées dans le modèle physique afin de simuler le comportement réel du stockage.

VII. Observations et analyse des résultats

Les simulations réalisées mettent en évidence des comportements très contrastés selon la stratégie de décision adoptée. Comme évoqué plus tôt, les stratégies à seuil fixe présentent une grande simplicité d'implémentation, mais peuvent conduire une inertie excessive. En effet, les prix subissant des formes de tendance et de saisonnalité, un seuil fixe (ne suivant donc pas ces tendances) n'amènerait qu'une consigne pendant une longue période. Par exemple, la forte demande en électricité l'hiver pour le chauffage, et l'été pour la climatisation, entraîne une hausse des prix et un dépassement du seuil impliquant une utilisation de l'énergie stockée pendant ces 2 saisons. Or, cela impliquerait qu'il faudrait avoir en sa possession un très grand nombre de CAES, remplis à 100%, pour répondre à ces demandes saisonnières. A l'inverse, le printemps et l'automne faisant montre de demandes moins importantes en électricité, le coût de celle-ci peut baisser jusqu'à passer en dessous du seuil.

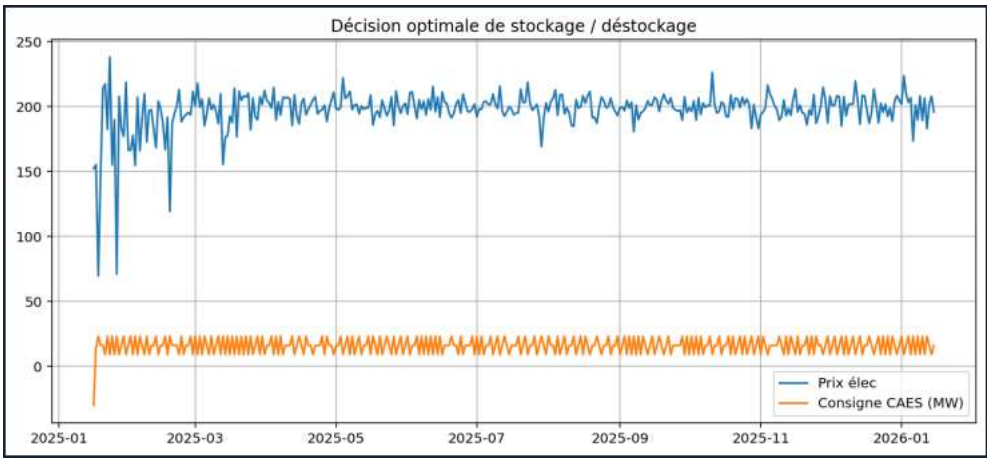


Figure 2 : Prévision du prix et consigne en seuil fixe

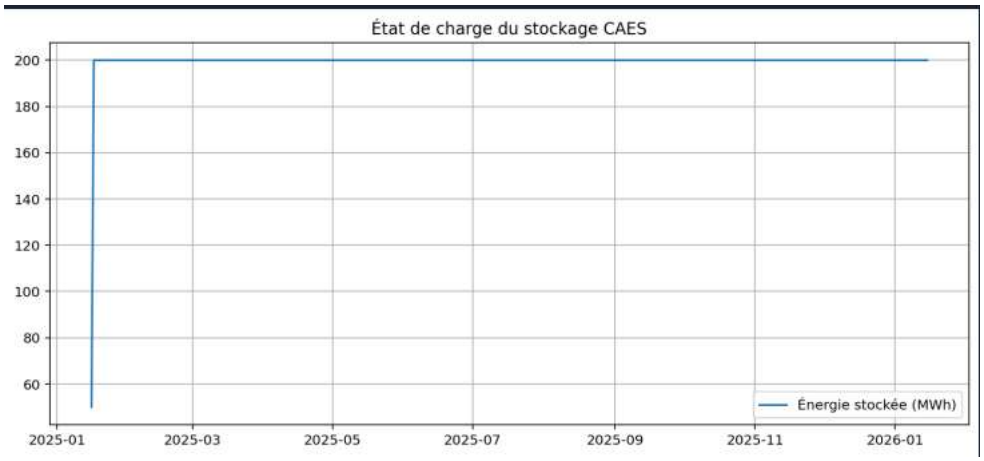


Figure 3 : Etat de stockage du CAES en seuil fixe

Les stratégies basées sur une utilisation des moyennes glissantes explicites des prix permettent une meilleure cohérence globale, mais leur performance dépend fortement de la qualité des prévisions. Les résultats illustrent ainsi le compromis fondamental entre robustesse et réactivité, qui constitue l'un des enjeux centraux du pilotage du stockage d'énergie sur les marchés.

a. **Résultats dans le cas adiabatique sur 30 jours**

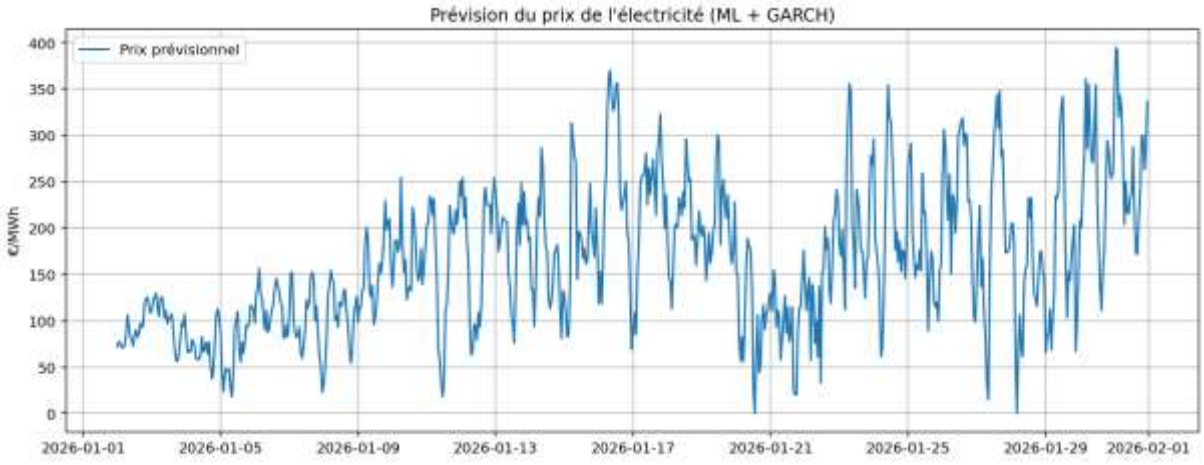


Figure 4a : Prévision prix électricité sur 30 jours

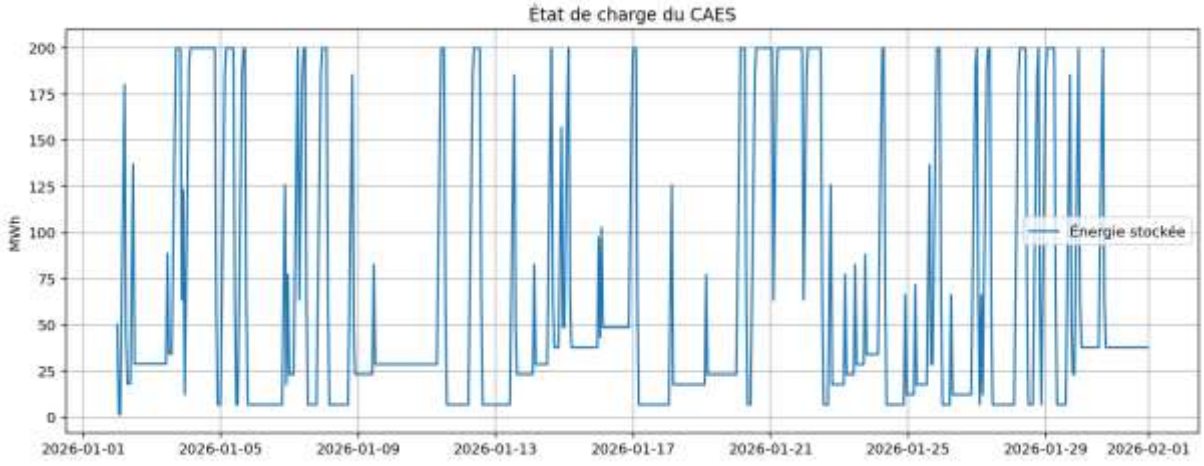


Figure 4b : Etat de stockage du CAES sur 30 jours

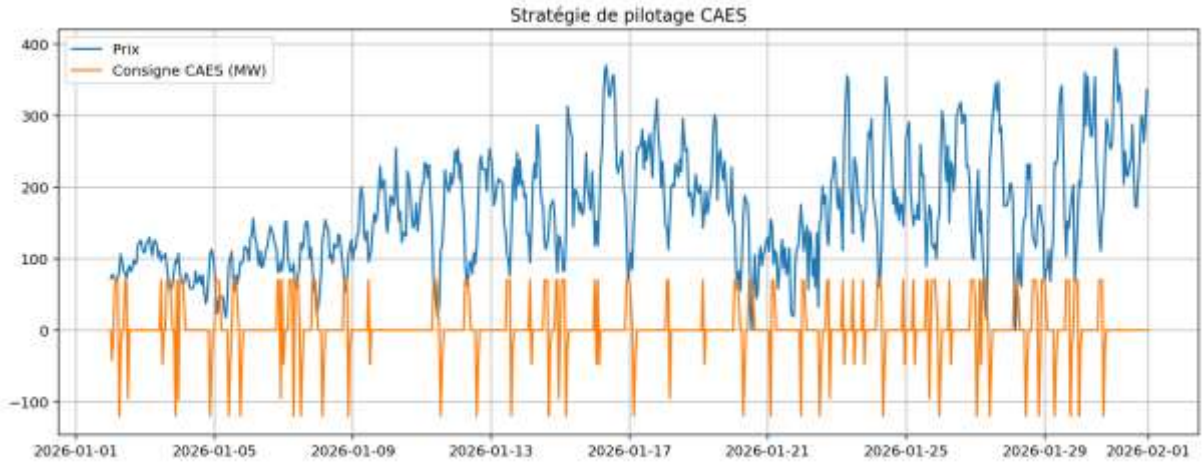


Figure 4c : Stratégie de pilotage du CAES sur 30 jours

b. **Résultats dans le cas isotherme sur 30 jours**

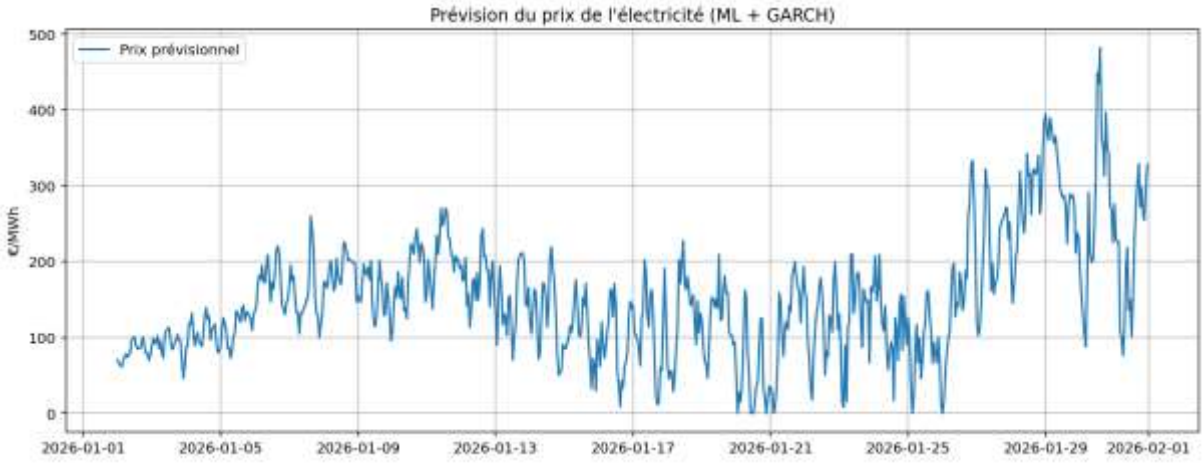


Figure 5a : Prévision prix électricité sur 30 jours

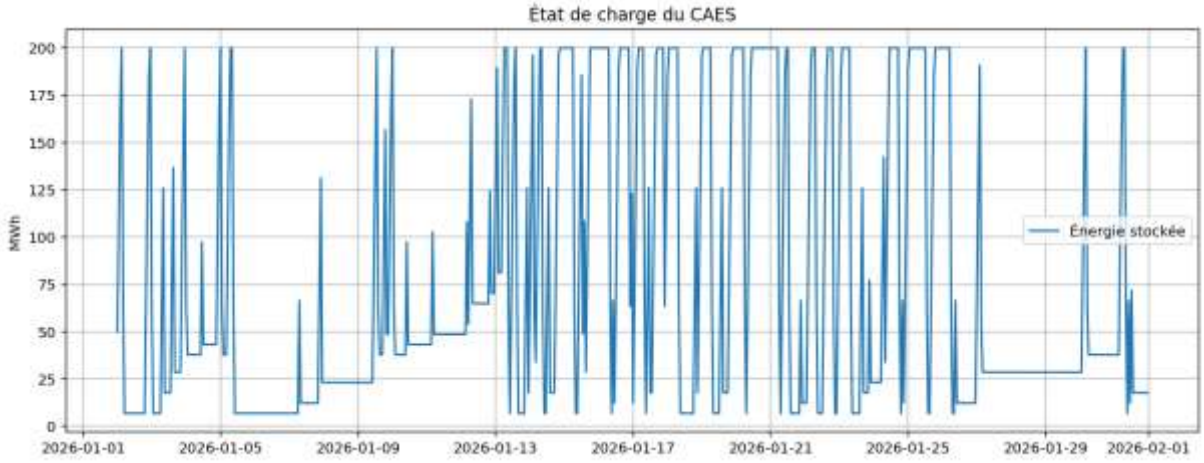


Figure 5b : Etat de stockage du CAES sur 30 jours

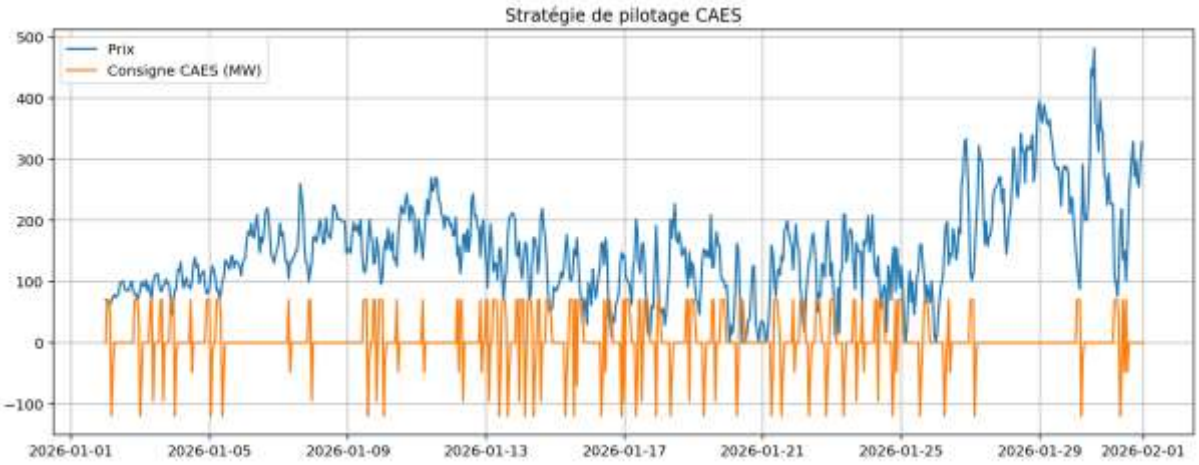


Figure 5c : Stratégie de pilotage du CAES sur 30 jours

c. **Résultats dans le cas adiabatique en intraday**

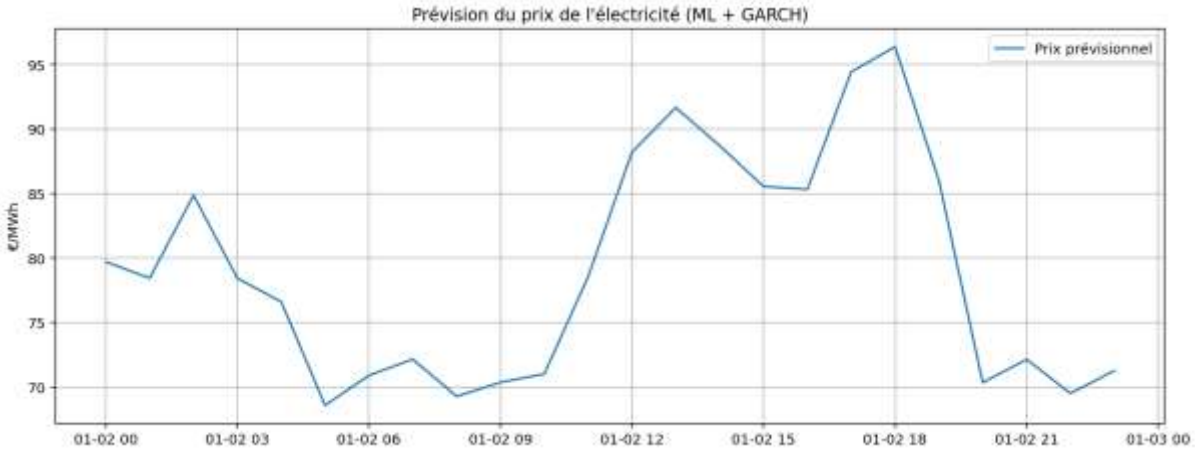


Figure 6a : Prévision prix électricité sur 1 jour

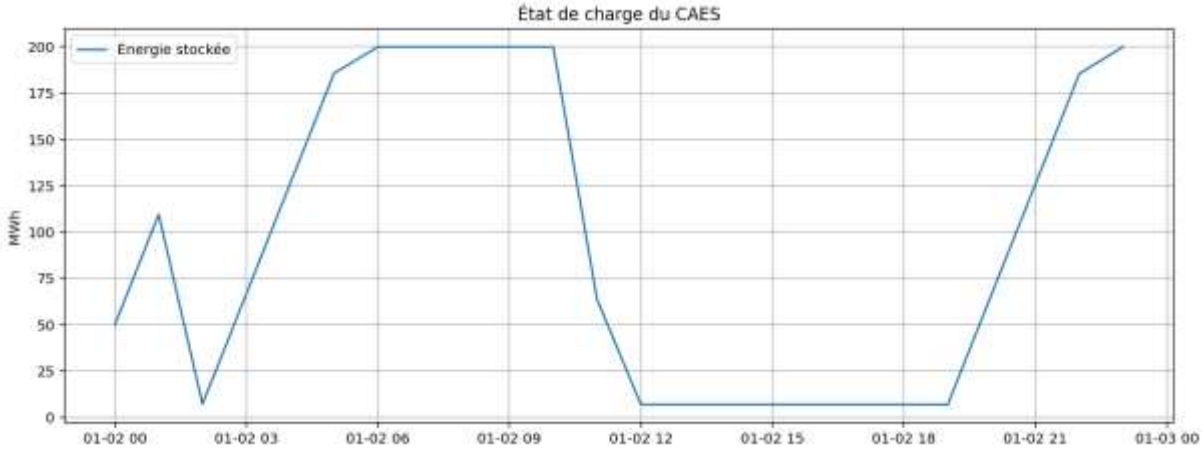


Figure 6b : Etat de stockage du CAES sur 1 jour

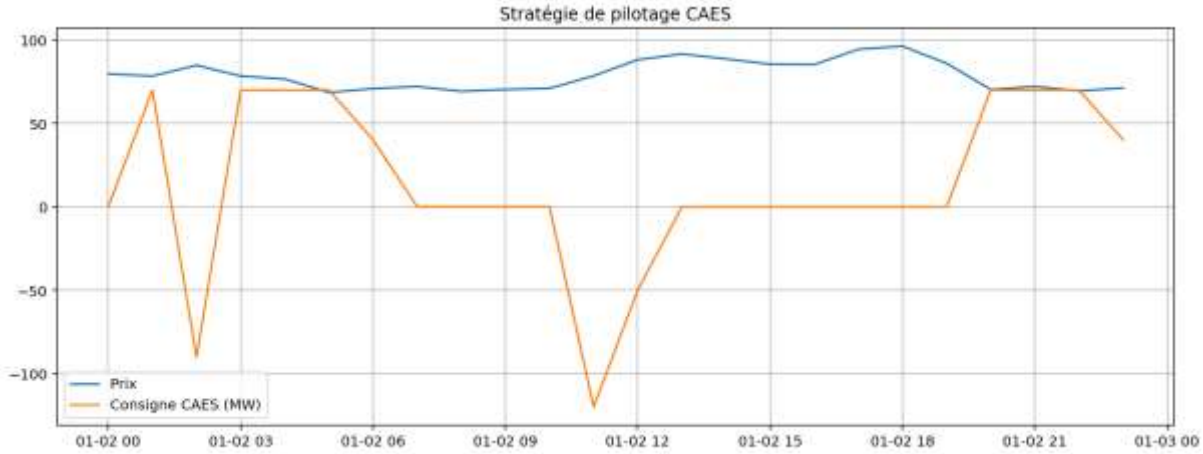


Figure 6c : Stratégie de pilotage du CAES sur 30 jours

d. **Résultats dans le cas isotherme en intraday**

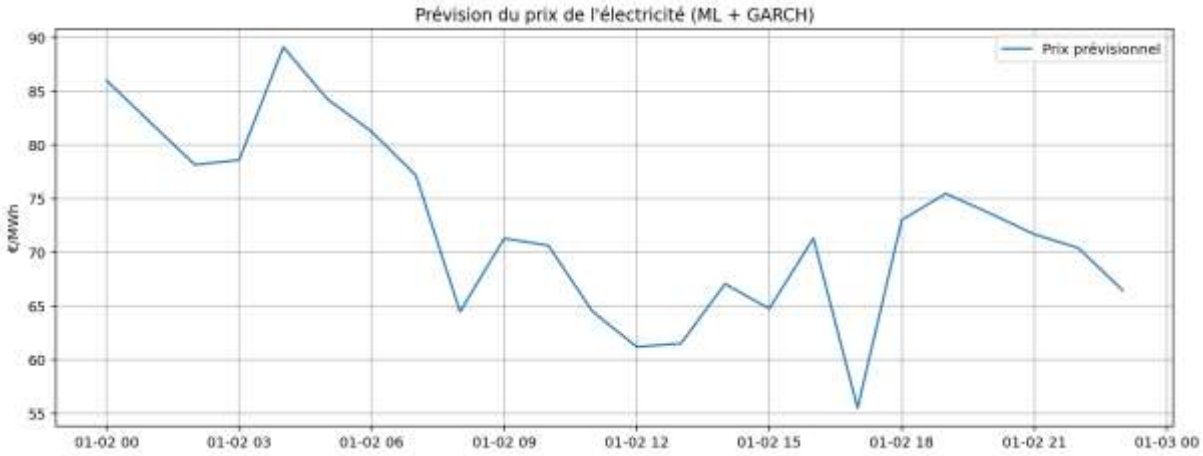


Figure 7a : Prévision prix électricité sur 1 jour

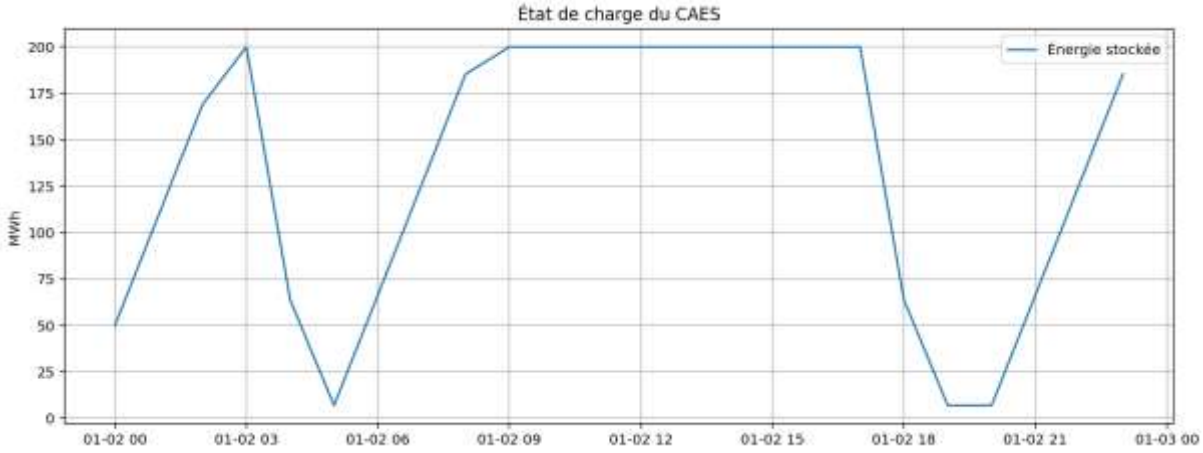


Figure 7b : Etat de stockage du CAES sur 1 jour

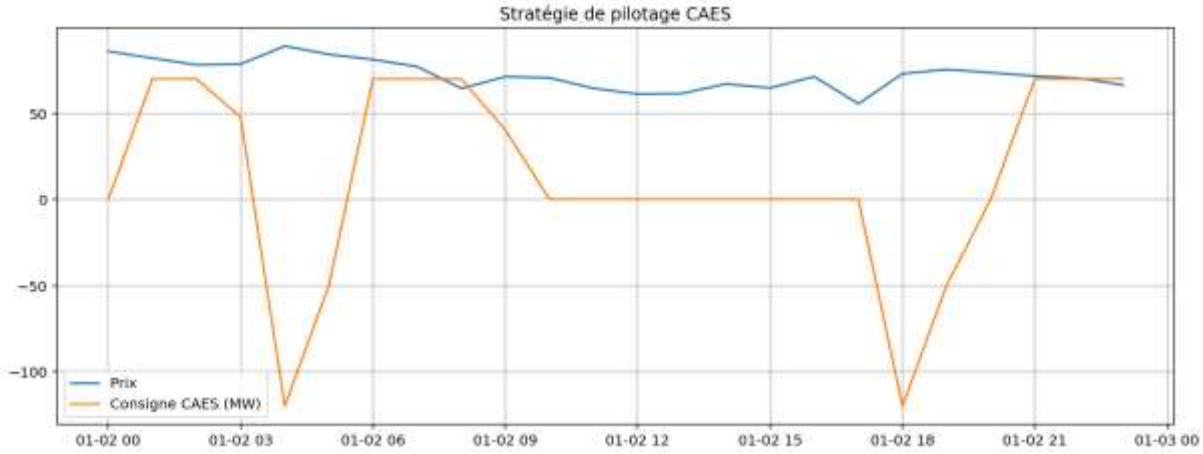


Figure 7c : Stratégie de pilotage du CAES sur 30 jours

Certaines croissances ou décroissances dans les graphes sont plus lentes que d'autres, car l'énergie est recalculée à chaque pas de temps avec une forme de proportionnalité avec le delta de temps durant lequel a lieu la consigne de charge ou décharge, ainsi qu'avec la puissance du compresseur/turbine à ce pas de temps.

Comme évoqué plus tôt dans le rapport, le manque de certaines données à intégrer en temps réel, comme le ferait un desk de trading, pour pouvoir utiliser un modèle 100% ML et ne pas avoir à avoir recours à une volatilité artificielle, fait que cela peut fausser certaines tendances et créer un cours de l'électricité très volatile sur le long terme (30 jours). Cependant, on remarque des résultats très intéressants en intraday, avec une évolution des prix réaliste et satisfaisante : stockage pendant la journée lorsque la demande est plus faible et achat sur le marché à prix plus faible ; utilisation de l'énergie stockée le matin et le soir lors des pics de consommation et de prix, qui corrobore ce que l'on appelle la « Duck Curve ».

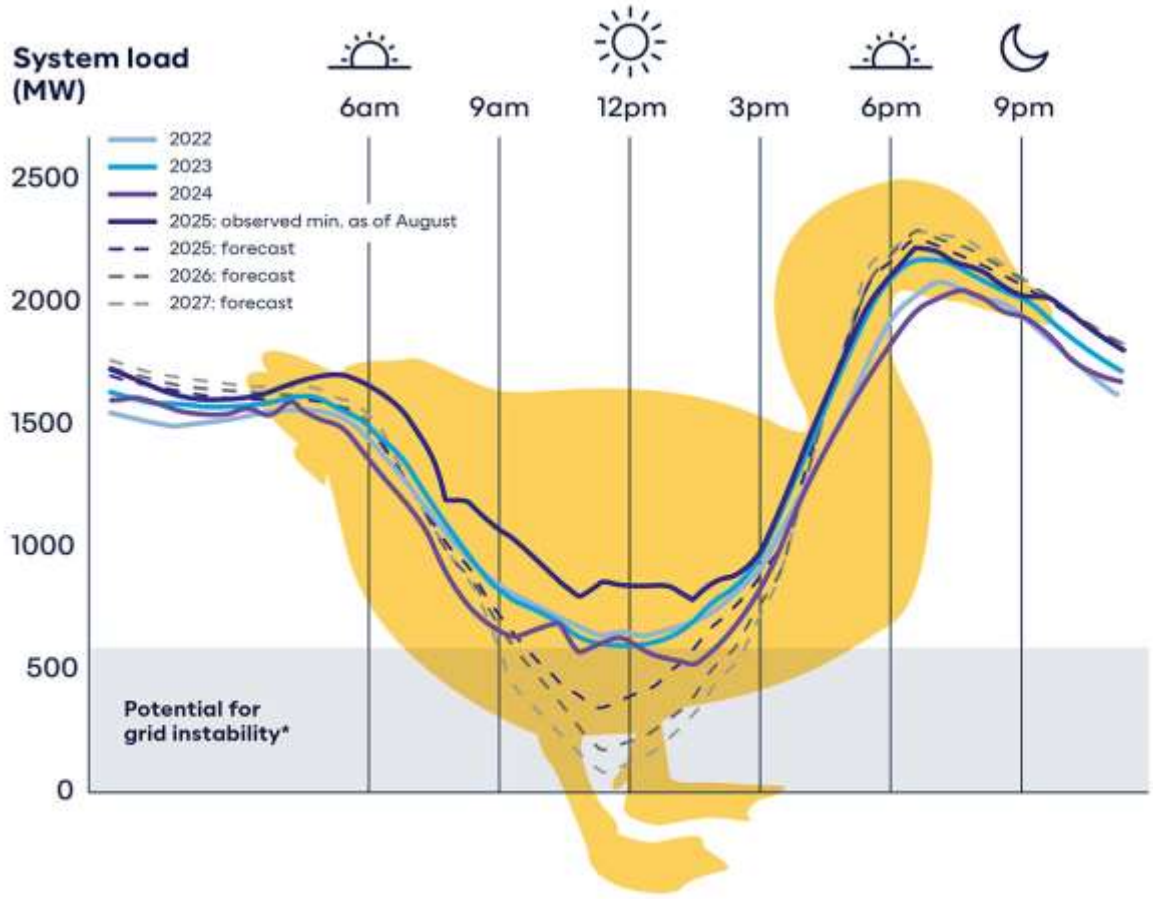


Figure 8 : Représentation de la « Duck Curve » sur 1 jour

VIII. Conclusion et perspectives

Ce travail a permis de développer une chaîne complète reliant le marché de l'électricité, la prévision des prix, les stratégies de décision et le modèle physique d'un système CAES. Il montre que l'optimisation du stockage ne peut être abordée uniquement sous l'angle technologique, mais doit intégrer pleinement la dimension économique et l'incertitude de marché.

La modélisation développée est satisfaisante, en particulier pour l'horizon de prévision souhaité, c'est-à-dire en intraday. Nous avons pu observer une recrudescence prévisible de la demande le matin et le soir (cf Duck Curve), ainsi qu'une saisonnalité poussant à une plus forte consommation d'énergie électrique l'été et l'hiver.

Il pourrait être intéressant, en prolongement de ce projet d'inclure des méthodes d'optimisation encore plus avancées, telles que la programmation dynamique ou l'apprentissage par renforcement, ainsi que l'extension du cadre à plusieurs horizons de marché. Ces pistes ouvrent la voie à une optimisation plus fine et séquentielle, notamment sur le long terme, et plus réaliste des systèmes de stockage dans le contexte énergétique européen.

IX. Annexes

```python
# Pricing_Elec_v4.py
# Electricity price forecasting framework
# - Random Forest regression
# - GARCH-based stochastic volatility

def charger_prix_electricite(path_csv: str, pays: str = "France")
    -> pd.Series:
    df = pd.read_csv(path_csv)

    df = df[df["Country"] == pays]

    df["Datetime (UTC)"] = pd.to_datetime(df["Datetime (UTC)"])
    df = df.set_index("Datetime (UTC)")

    prix = df["Price (EUR/MWhe)"].sort_index()

    return prix

def construire_features(prix: pd.Series, nb_lags: int = 30):

    df = pd.DataFrame({"y": prix})

    for lag in range(1, nb_lags + 1):
        df[f"lag_{lag}"] = df["y"].shift(lag)

    df["hour"] = df.index.hour
    df["dayofweek"] = df.index.dayofweek
    df["month"] = df.index.month

    df["hour_sin"] = np.sin(2 * np.pi * df["hour"] / 24)
    df["hour_cos"] = np.cos(2 * np.pi * df["hour"] / 24)

    df["dow_sin"] = np.sin(2 * np.pi * df["dayofweek"] / 7)
    df["dow_cos"] = np.cos(2 * np.pi * df["dayofweek"] / 7)

    df["month_sin"] = np.sin(2 * np.pi * df["month"] / 12)
    df["month_cos"] = np.cos(2 * np.pi * df["month"] / 12)

    df = df.dropna()

    X = df.drop(columns="y")
    y = df["y"]

    return X, y

def entrainer_modele(X, y):

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, shuffle=False
    )

    model = RandomForestRegressor(
        n_estimators=300,
        max_depth=15,
        random_state=42,
        n_jobs=-1
    )
```

```python

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    mae = mean_absolute_error(y_test, y_pred)


    return model

def ajuster_garch(prix: pd.Series):

    rendements = prix.diff().dropna()

    model_garch = arch_model(
        rendements,
        vol="Garch",
        p=1,
        q=1,
        mean="Zero",
        dist="normal"
    )

    res = model_garch.fit(disp="off")

    return res

def prevoir_prix(
    model,
    garch_res,
    prix: pd.Series,
    horizon_h: int = 8760,
    nb_lags: int = 30
):

    predictions = []

    prix_moyen = prix.mean()
    alpha_reversion = 0.05

    historique = prix.iloc[-nb_lags:].tolist()
    current_time = prix.index[-1]

    sigma = np.sqrt(
        garch_res.forecast(horizon=horizon_h).variance.values[-1]
    )

    for h in range(horizon_h):

        current_time += pd.Timedelta(hours=1)

        features = {
            f"lag_{lag}": historique[-lag]
            for lag in range(1, nb_lags + 1)
        }

        hour = current_time.hour
        dow = current_time.dayofweek
```

```
        month = current_time.month

        features.update({
            "hour_sin": np.sin(2 * np.pi * hour / 24),
            "hour_cos": np.cos(2 * np.pi * hour / 24),
            "dow_sin": np.sin(2 * np.pi * dow / 7),
            "dow_cos": np.cos(2 * np.pi * dow / 7),
            "month_sin": np.sin(2 * np.pi * month / 12),
            "month_cos": np.cos(2 * np.pi * month / 12)
        })

        X_pred = pd.DataFrame([features])

        pred = model.predict(X_pred)[0]
        pred += np.random.normal(0, sigma[h])
        pred += alpha_reversion * (prix_moyen - pred)

        pred = np.clip(pred, 0, 500)

        predictions.append(pred)
        historique.append(pred)
        historique.pop(0)

    index_pred = pd.date_range(
        start=prix.index[-1] + pd.Timedelta(hours=1),
        periods=horizon_h,
        freq="H"
    )

    return pd.Series(predictions, index=index_pred)
```

Listing 1: Pricing Elec v4.py

```python
# -*- coding: utf-8 -*-

from dataclasses import dataclass
import numpy as np
import pandas as pd

@dataclass
class ParametresCAESAdiabatique:
    volume_caverne_m3: float = 5e5
    pression_min_bar: float = 1.0
    pression_max_bar: float = 70.0
    temperature_ambiante_K: float = 293.15
    gamma: float = 1.4
    R: float = 287.0
    fenetre_heures: int = 720
    puissance_compresseur_mw: float = 70.0
    puissance_turbine_mw: float = 120.0
    capacite_stockage_mwh: float = 200.0
    rendement_charge: float = 0.85
    rendement_decharge: float = 0.88
    pertes_horaires: float = 1e-4
    energie_initiale_mwh: float = 50.0


def simuler_caes_adiabatique(index, consigne_MW, p:
    ParametresCAESAdiabatique):

    dt_h = (index[1] - index[0]).total_seconds() / 3600

    n = len(index)

    P = np.zeros(n)
    T = np.zeros(n)
    m = np.zeros(n)

    energie = np.zeros(n)

    P[0] = p.pression_min_bar * 1e5
    T[0] = p.temperature_ambiante_K
    m[0] = P[0] * p.volume_caverne_m3 / (p.R * T[0])
    energie[0] = p.energie_initiale_mwh

    W_comp = np.zeros(n)
    W_turb = np.zeros(n)
    W_reseau = np.zeros(n)

    for t in range(1, n):


        P[t], T[t], m[t], energie[t] = P[t-1], T[t-1], m[t-1],
            energie[t-1]
        P[t] *= (1 - p.pertes_horaires * dt_h)

        consigne = consigne_MW.iloc[t]

        #Compression
        if consigne > 0:
```

```
56
57              P_elec = min(consigne, p.puissance_compresseur_mw)
58              P_elec_eff = P_elec * p.rendement_charge
59
60              energie[t] += P_elec_eff * dt_h
61              energie[t] = min(energie[t], p.capacite_stockage_mwh)
62
63              W_comp[t] = P_elec
64              W_reseau[t] = +P_elec
65
66          # D tente
67          elif consigne < 0:
68
69              P_elec = min(-consigne, p.puissance_turbine_mw)
70
71              P_elec_eff = P_elec / p.rendement_decharge
72
73              energie[t] -= P_elec_eff * dt_h
74              energie[t] = max(energie[t], 0)
75
76              W_turb[t] = P_elec
77              W_reseau[t] = -P_elec
78
79          # Mise    jour pression fictive
80          taux = energie[t] / p.capacite_stockage_mwh
81          P[t] = (p.pression_min_bar + taux * (p.pression_max_bar - p
                .pression_min_bar)) * 1e5
82
83      return pd.DataFrame(
84          {
85              "pression_bar": P / 1e5,
86              "temperature_C": T - 273.15,
87              "masse_air_kg": m,
88              "energie_stockee_mwh": energie,
89              "P_comp_MW": W_comp,
90              "P_turb_MW": W_turb,
91              "P_reseau_MW": W_reseau,
92          },
93          index=index,
94      )
```

Listing 1: CAES Model Adiabatique.py

```python
# -*- coding: utf-8 -*-

from dataclasses import dataclass
import numpy as np
import pandas as pd


@dataclass
class ParametresCAESIsotherme:
    volume_caverne_m3: float = 5e5
    pression_min_bar: float = 1.0
    pression_max_bar: float = 70.0

    temperature_K: float = 293.15
    R: float = 287.0

    puissance_compresseur_mw: float = 70.0
    puissance_turbine_mw: float = 120.0

    rendement_charge: float = 0.85
    rendement_decharge: float = 0.88

    capacite_stockage_mwh: float = 200.0
    energie_initiale_mwh: float = 50.0

    pertes_horaires: float = 5e-5


def simuler_caes_isotherme(index, consigne_MW, p:
    ParametresCAESIsotherme):

    dt_h = (index[1] - index[0]).total_seconds() / 3600
    n = len(index)


    P = np.zeros(n)
    m = np.zeros(n)


    energie = np.zeros(n)


    P[0] = p.pression_min_bar * 1e5
    m[0] = P[0] * p.volume_caverne_m3 / (p.R * p.temperature_K)
    energie[0] = p.energie_initiale_mwh


    W_comp = np.zeros(n)
    W_turb = np.zeros(n)
    W_reseau = np.zeros(n)

    for t in range(1, n):

        P[t], m[t], energie[t] = P[t-1], m[t-1], energie[t-1]

        energie[t] *= (1 - p.pertes_horaires * dt_h)

```

```
        consigne = consigne_MW.iloc[t]

        # Compression isotherme
        if consigne > 0:

            P_elec = min(consigne, p.puissance_compresseur_mw)
            P_elec_eff = P_elec * p.rendement_charge

            energie[t] += P_elec_eff * dt_h
            energie[t] = min(energie[t], p.capacite_stockage_mwh)

            W_comp[t] = P_elec
            W_reseau[t] = +P_elec

        # D tente isotherme
        elif consigne < 0:

            P_elec = min(-consigne, p.puissance_turbine_mw)
            P_elec_eff = P_elec / p.rendement_decharge

            energie[t] -= P_elec_eff * dt_h
            energie[t] = max(energie[t], 0)

            W_turb[t] = P_elec
            W_reseau[t] = -P_elec


        taux = energie[t] / p.capacite_stockage_mwh
        P[t] = (p.pression_min_bar + taux * (p.pression_max_bar - p
            .pression_min_bar)) * 1e5

        m[t] = P[t] * p.volume_caverne_m3 / (p.R * p.temperature_K)

    return pd.DataFrame(
        {
            "pression_bar": P / 1e5,
            "masse_air_kg": m,
            "energie_stockee_mwh": energie,
            "P_comp_MW": W_comp,
            "P_turb_MW": W_turb,
            "P_reseau_MW": W_reseau,
        },
        index=index,
    )
```

Listing 1: CAES Model Isotherme.py

```python
# -*- coding: utf-8 -*-
from __future__ import annotations

"""
Modele de decision sur la base d'un seuil egal a la moyenne mobile des prix, ce qui provoque beaucoup de variations dans les consignes
"""

from dataclasses import dataclass
from typing import Dict, Tuple, Optional

import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from sklearn.ensemble import RandomForestClassifier


@dataclass
class ParametresDecisionBis:
    fenetre_heures: int = 720
    puissance_compresseur_mw: float = 70.0
    puissance_turbine_mw: float = 120.0
    capacite_stockage_mwh: float = 200.0
    rendement_charge: float = 0.80
    rendement_decharge: float = 0.85
    pertes_horaires_fraction: float = 0.0005
    energie_initiale_mwh: float = 50.0


def decisions_par_seuil_glissant(prix: pd.Series,
                                 param: ParametresDecisionBis) -> pd.DataFrame:
    index = prix.index
    dt_heures = (index[1] - index[0]).total_seconds() / 3600.0

    fenetre = int(param.fenetre_heures)
    seuil = prix.rolling(window=fenetre, min_periods=1).mean()

    energie = np.zeros(len(index))
    energie[0] = np.clip(param.energie_initiale_mwh, 0, param.capacite_stockage_mwh)
    charge = np.zeros(len(index))
    decharge = np.zeros(len(index))

    for t in range(1, len(index)):
        e = energie[t-1] * (1.0 - param.pertes_horaires_fraction * dt_heures)

        p = prix.iloc[t]
        p_seuil = seuil.iloc[t]

        if p < p_seuil and e < param.capacite_stockage_mwh:

            p_c = param.puissance_compresseur_mw
```

```
53
54              e_max_add = (param.capacite_stockage_mwh - e)
55              p_c = min(p_c, e_max_add / (dt_heures * param.
                    rendement_charge))
56              delta_e = p_c * dt_heures * param.rendement_charge
57              e = e + delta_e
58              charge[t] = p_c
59
60          elif p > p_seuil and e > 0:
61
62              p_d = param.puissance_turbine_mw
63
64              e_max_prelevee = e
65              p_d = min(p_d, (e_max_prelevee * param.
                    rendement_decharge) / dt_heures)
66              e_prelevee = (p_d * dt_heures) / max(param.
                    rendement_decharge, 1e-9)
67              e = e - e_prelevee
68              decharge[t] = p_d
69
70
71          energie[t] = np.clip(e, 0, param.capacite_stockage_mwh)
72
73      df = pd.DataFrame(
74          {
75              "prix_eur_mwh": prix.values,
76              "seuil_glissant": seuil.values,
77              "charge_caes_mw": charge,
78              "decharge_caes_mw": decharge,
79              "energie_stockee_mwh": energie,
80          },
81          index=index,
82      )
83      return df
```

Listing 1: Modèle de Décision avec Seuil Glissant.py

```python
# -*- coding: utf-8 -*-

"""
Pilotage principal CAES
- Pricing ML + GARCH (Pricing_Elec_v4)
- Decision par seuil glissant
- Simulation CAES
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from Pricing_Elec_v4 import (
    charger_prix_electricite,
    construire_features,
    entrainer_modele,
    ajuster_garch,
    prevoir_prix
)

from CAES_Model_Adiabatique import ParametresCAESAdiabatique,
    simuler_caes_adiabatique      #idem pour l'isotherme en
    remplacant adiabatique par isotherme
from Modele_Decision_Seuil_Glissant import (
    ParametresDecisionBis,
    decisions_par_seuil_glissant
)


# Parametres
DATE_DEBUT = "2022-01-01"
DATE_FIN   = "2026-01-01"

HORIZON_PREVISION_JOURS = 30
HORIZON_PREVISION_HEURES = HORIZON_PREVISION_JOURS * 24

PATH_PRIX = r"C:\Users\mehdi\Documents\Mehdi\ENSAM\PJE_BIS_ENSAM\
    France.csv"

prix_hist = charger_prix_electricite(PATH_PRIX, pays="France")

prix_hist = prix_hist.loc[DATE_DEBUT:DATE_FIN]
prix_hist = prix_hist.asfreq("H")

# Securite : suppression trous eventuels
prix_hist = prix_hist.interpolate()

print(f"Historique charge : {len(prix_hist)} heures")

# Entrainement pricing

X, y = construire_features(prix_hist)

model_prix = entrainer_modele(X, y)

garch_res = ajuster_garch(prix_hist)
```

```python


# Prevision prix

prix_prevu = prevoir_prix(
    model=model_prix,
    garch_res=garch_res,
    prix=prix_hist,
    horizon_h=HORIZON_PREVISION_HEURES
)



# Modele decision

param_decision = ParametresDecisionBis(
    fenetre_heures=720,                # 30 jours glissants
    capacite_stockage_mwh=200.0,
    puissance_compresseur_mw=70.0,
    puissance_turbine_mw=120.0,
    energie_initiale_mwh=50.0,
)

df_decision = decisions_par_seuil_glissant(
    prix_prevu,
    param_decision
)

consigne_caes = (
    df_decision["charge_caes_mw"]
    - df_decision["decharge_caes_mw"]
)



# Simulation CAES

param_caes = ParametresCAESAdiabatique() #ou
    ParametresCAESIsotherme

df_caes = simuler_caes_adiabatique(   #ou simuler_caes_isotherme
    index=prix_prevu.index,
    consigne_MW=consigne_caes,
    p=param_caes
)



# Visualisation

plt.figure(figsize=(14, 5))
plt.plot(prix_prevu, label="Prix previsionnel")
plt.title("Prevision du prix de l'electricite (ML + GARCH)")
plt.ylabel("EUR/MWh")
plt.grid()
plt.legend()
```

```
111 plt.show()
112
113
114 plt.figure(figsize=(14, 5))
115 plt.plot(df_caes.index, df_caes["energie_stockee_mwh"], label="
        Energie stockee")
116 plt.title("Etat de charge du CAES")
117 plt.ylabel("MWh")
118 plt.grid()
119 plt.legend()
120 plt.show()
121
122
123 plt.figure(figsize=(14, 5))
124 plt.plot(prix_prevu, label="Prix")
125 plt.plot(consigne_caes, label="Consigne CAES (MW)")
126 plt.title("Strategie de pilotage CAES")
127 plt.grid()
128 plt.legend()
129 plt.show()
```

Listing 1: Pilotage de la Stratégie de Stockage (CAES Adiabatique/Isotherme)

```python
# -*- coding: utf-8 -*-
from __future__ import annotations

"""
Modele de decision sur la base d'un seuil fixe, ce qui provoque peu de variations dans les consignes
"""
from dataclasses import dataclass
from typing import Dict, Tuple, Optional

import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from sklearn.ensemble import RandomForestClassifier


@dataclass
class ParametresDecision:
    fenetre_heures: int = 720
    puissance_compresseur_mw: float = 70.0
    puissance_turbine_mw: float = 120.0
    capacite_stockage_mwh: float = 200.0
    rendement_charge: float = 0.85
    rendement_decharge: float = 0.88
    pertes_horaires_fraction: float = 5e-5
    energie_initiale_mwh: float = 50.0


def construire_features_decision(df: pd.DataFrame, nb_lags: int = 24) -> Tuple[pd.DataFrame, pd.Series]:
    X = pd.DataFrame(index=df.index)
    X["prix"] = df["prix_eur_mwh"]
    X["energie_stockee"] = df["energie_stockee_mwh"]
    X["demande"] = df["demande_mw"]

    X["heure"] = df.index.hour
    X["jour_semaine"] = df.index.dayofweek
    X["weekend"] = (df.index.dayofweek >= 5).astype(int)
    X["sin_heure"] = np.sin(2 * np.pi * X["heure"] / 24.0)
    X["cos_heure"] = np.cos(2 * np.pi * X["heure"] / 24.0)

    for k in range(1, nb_lags + 1):
        X[f"lag_prix_{k}"] = df["prix_eur_mwh"].shift(k)

    y = df["action"].astype(int)
    donnees = pd.concat([X, y.rename("cible")], axis=1).dropna()
    return donnees.drop(columns=["cible"]), donnees["cible"]


def calculer_actions_oracle_cvxpy(prix: pd.Series, demande_mw: pd.Series, param: ParametresDecision, energie_initiale_mwh: float) -> pd.DataFrame:

    try:
        import cvxpy as cp
```

```
    except ImportError as e:
        raise ImportError("Installe cvxpy pour utiliser l'oracle :
            pip install cvxpy") from e

    index = prix.index
    dt_heures = (index[1] - index[0]).total_seconds() / 3600.0
    n = len(index)

    achat = cp.Variable(n, nonneg=True)
    charge = cp.Variable(n, nonneg=True)
    decharge = cp.Variable(n, nonneg=True)
    energie = cp.Variable(n)

    contraintes = []

    contraintes += [charge <= param.puissance_compresseur_mw]
    contraintes += [decharge <= param.puissance_turbine_mw]

    contraintes += [energie >= 0]
    contraintes += [energie <= param.capacite_stockage_mwh]

    pertes = (1.0 - param.pertes_horaires_fraction * dt_heures)
    contraintes += [energie[0] == energie_initiale_mwh]
    for t in range(1, n):
        contraintes += [
            energie[t] == energie[t - 1] * pertes
            + charge[t] * dt_heures * param.rendement_charge
            - (decharge[t] * dt_heures) / max(param.
                rendement_decharge, 1e-9)
        ]

    contraintes += [achat + decharge == demande_mw.values + charge]

    # Minimise cout d'achat
    cout = cp.sum(cp.multiply(prix.values, achat) * dt_heures)
    #On ajoute les lignes suivantes si l'on souhaite pousser un
        mod le valorisant la charge
    # et on prend achat - decharge dans la fonction de cout au lieu
         d'achat tout seul
    #c_cycle = 1.0
    #cout += c_cycle * cp.sum(charge * dt_heures + decharge *
        dt_heures)
    #E_cible = 50.0
    #penalite_finale = 0.05
    #cout += penalite_finale * cp.square(energie[-1] - E_cible)
    #E_max_oper = 0.9 * param.capacite_stockage_mwh
    #contraintes += [energie <= E_max_oper]

    probleme = cp.Problem(cp.Minimize(cout), contraintes)
    probleme.solve(solver=cp.ECOS)



    df = pd.DataFrame(
        {
            "prix_eur_mwh": prix.values,
            "demande_mw": demande_mw.values,
```

```
            "achat_reseau_mw": achat.value,
            "charge_caes_mw": charge.value,
            "decharge_caes_mw": decharge.value,
            "energie_stockee_mwh": energie.value,
        }, index=index,)


    seuil = 1e-3
    action = np.zeros(n, dtype=int)
    action[df["charge_caes_mw"].values > seuil] = 1
    action[df["decharge_caes_mw"].values > seuil] = 2
    df["action"] = action
    return df


def entrainer_modele_decision(df_oracle: pd.DataFrame) ->
    RandomForestClassifier:
    X, y = construire_features_decision(df_oracle, nb_lags=24)
    X_train, X_test, y_train, y_test = train_test_split(X, y,
        test_size=0.2, shuffle=False)

    modele = RandomForestClassifier(n_estimators=300, random_state
        =0, class_weight="balanced_subsample", min_samples_leaf=5)

    modele.fit(X_train, y_train)
    pred = modele.predict(X_test)

    print(classification_report(y_test, pred, digits=3))
    return modele


def appliquer_modele_en_ligne(modele: RandomForestClassifier,
    df_etat: pd.DataFrame) -> pd.Series:

    X, _ = construire_features_decision(df_etat.assign(action=0),
        nb_lags=24)
    pred = modele.predict(X)
    return pd.Series(pred, index=X.index, name="action_predite")


if __name__ == "__main__":
    index = pd.date_range("2024-01-01", periods=24*60, freq="h", tz
        ="Europe/Paris")
    rng = np.random.default_rng(0)

    # Exemple prix
    prix = pd.Series(70 + 20*np.sin(2*np.pi*index.hour/24) + rng.
        normal(0, 5, len(index)), index=index)

    # Exemple demande
    demande = pd.Series(30 + 10*((index.hour>=18) & (index.hour
        <=23)).astype(int), index=index)

    param = ParametresDecision()
    df_oracle = calculer_actions_oracle_cvxpy(prix, demande, param,
         energie_initiale_mwh=50.0)

```

```
    modele = entrainer_modele_decision(df_oracle)

    actions_pred = appliquer_modele_en_ligne(modele, df_oracle[["
        prix_eur_mwh","energie_stockee_mwh","demande_mw","action"
        ]].copy())
    print(actions_pred.head())
```

Listing 1: Modele de Decision par Seuil Fixe

X. Bibliographie

[1] https://energystorageeurope.eu/wp-content/uploads/2016/03/EASE_TD_ACAES.pdf

[2] https://www.dropbox.com/scl/fi/051ielwkjfcotus2h4vyk/ENEA-Consulting-Le-Stockage-dEnergie1.pdf?rlkey=q58z3vcf6otdctnhcf4z59pje&e=1&dl=0

[3] https://www.synergy.net.au/Blog/2021/10/Everything-you-need-to-know-about-the-Duck-Curve

[4] https://forecastdocs.modoenergy.com/docs/intraday-model

[5] https://ember-energy.org/data/european-wholesale-electricity-price-data/

[6] https://fr.wikipedia.org/wiki/Stockage_d%27%C3%A9nergie_par_air_comprim%C3%A9

[7] https://www.consilium.europa.eu/en/infographics/how-is-eu-electricity-produced-and-sold

[8] https://www.marketdataforecast.com/market-reports/europe-power-market

[9] Cours de thermodynamique, ENSAM